

Student Mental Wellness Check-in System

Course: Introduction to Computing and Programming

Instructor: Bibi Sara Karimullah

Group: 06

Student Names: Ekhing, Chelsi, Khing Me

Submission Date: August 13, 2026

1. Abstract

The **Student Mental Wellness Check-in System** is a simple C program designed to check a student's general wellness based on mood, stress level, sleep hours, and study hours. The program calculates a wellness score and displays a status such as Excellent, Good, Fair, or Needs Attention. It also provides suggestions based on the student's condition. The project was developed using the C programming concepts we learned in class, such as functions, loops, conditional statements, variables, and user input.

2. Introduction

Student mental wellness is important because stress, lack of sleep, and excessive studying can affect students' daily lives and academic performance. We chose this topic because these are common issues that students can relate to.

The main objective of our project is to create a simple system where students can enter some information about their daily wellness and receive a basic wellness score and suggestions.

The system focuses on four factors:

- Mood
- Stress level
- Sleep hours
- Study hours

The project is not a professional mental health assessment. It is a simple programming project that demonstrates how programming can be used to address a relatable student issue.

3. Methodology / Implementation

We developed the project step by step using the C programming concepts covered in our course.

Step 1: Taking student information

First, we created variables to store the student's name, ID, mood, stress, sleep hours, and study hours.

For example:

```
char name[50];
int id;
int mood = 0, stress = 0;
int sleep, study;
int score;
char choice;
```

Explanation:

We used a character array for the student's name because the name is a string. The other information is stored using integer variables because they are numerical values.

Step 2: Taking preset student information

We included two preset students to demonstrate how the system works with different conditions. For example:

```
strcpy(name, "Tuly");  
id = 242190;  
mood = 4;  
stress = 2;  
sleep = 7;  
study = 5;
```

The first preset student has generally healthy values.

The second preset student has:

```
strcpy(name, "Ekhing");  
id = 242304;  
mood = 2;  
stress = 5;  
sleep = 5;  
study = 9;
```

These values represent a student with lower mood, high stress, less sleep, and long study hours.

Explanation:

We used preset students so that we could demonstrate two different outcomes without entering the information manually every time. The first student demonstrates a healthy condition, while the second demonstrates a situation where the student needs more attention.

Step 3: Input validation

For mood and stress, we used a while loop to make sure the user enters a value between 1 and 5.

```

while(mood < 1 || mood > 5)
{
    printf("Enter Mood (1-5): ");
    scanf("%d", &mood);

    if(mood < 1 || mood > 5)
    {
        printf("Invalid input! Please enter a number between 1 and 5.\n");
    }
}

```

Explanation:

The while loop continues when the entered value is invalid. If the user enters a number from 1 to 5, the loop ends. If the number is outside this range, the program displays an error message and asks again.

Step 4: Calculating the wellness score

We created a separate function called calculateScore().

```

int calculateScore(int mood, int stress, int sleep, int study)
{
    int score = 100;

    if(mood <= 2)
    {
        score -= 20;
    }

    if(stress >= 4)
    {
        score -= 20;
    }

    if(sleep < 6)
    {
        score -= 20;
    }

    if(study > 8)
    {
        score -= 10;
    }

    if(score < 0)
    {
        score = 0;
    }

    return score;
}

```

Explanation:

The score starts from 100. Points are deducted when the student's values show possible problems. For example, low mood deducts 20 points, high stress deducts 20

points, less than six hours of sleep deducts 20 points, and studying for more than eight hours deducts 10 points.

Step 5: Displaying the status

After calculating the score, we use if-else statements to determine the student's status.

```
if(score >= 80)
{
    printf("Status      : Excellent\n");
}
else if(score >= 60)
{
    printf("Status      : Good\n");
}
else if(score >= 40)
{
    printf("Status      : Fair\n");
}
else
{
    printf("Status      : Needs Attention\n");
}
```

Explanation:

The final score is used to place the student into one of four categories. This makes the result easier for the user to understand.

Step 6: Providing suggestions

The suggestions() function checks the student's conditions and prints appropriate suggestions. For example:

```
if(mood <= 2)
{
    printf("- Try doing something you enjoy.\n");
}
```

Explanation:

The function checks each wellness factor and provides a suggestion when a problem is detected. If the student has healthy values, the program displays a positive message.

4. Tools and Technologies C Programming Language

We used C because it is the programming language taught in our course. The project allowed us to apply the concepts we learned in class to a practical problem.

Online C Compiler

We used an **online C compiler** to quickly test small sections of our code and check whether the program was working correctly. It was useful for testing the logic before putting everything together.

CS50.dev

We used **CS50.dev** as our main coding environment while developing and testing the project. It provided a convenient environment for writing, compiling, and running our C program.

Class Notes

We used our **class notes** to understand and apply the programming concepts taught in the course. These included functions, if-else statements, while loops, variables, strings, `scanf()`, `printf()`, and `fgets()`.

5. Results / Output

The program successfully takes student information, calculates a wellness score, displays the wellness status, and provides suggestions.

For example, the first preset student receives a high wellness score because the values entered are within the healthy range.

The second preset student receives a lower score because the values include low mood, high stress, less sleep, and long study hours.

Healthy Student Output

```
=====
Student Mental Wellness Check-in
=====

Student Name   : Tuly
Student ID    : 242190
Mood          : 4
Stress Level  : 2
Sleep Hours   : 7
Study Hours   : 5
Wellness Score : 100
Status        : Excellent

Suggestions:
- Great! Keep maintaining your healthy routine.
```

Needs Attention Output

```
=====
Student Mental Wellness Check-in
=====
```

```
Student Name   : Ekhing
Student ID    : 242304
Mood          : 2
Stress Level   : 5
Sleep Hours   : 5
Study Hours    : 9
Wellness Score : 30
Status        : Needs Attention
```

Suggestions:

- Try doing something you enjoy.
- Take regular breaks while studying.
- Try to sleep at least 7-8 hours.
- Avoid studying continuously for long hours.

User Input and Validation

```
Would you like to check another student? (Y/N): Y

=====
Student Mental Wellness Check-in
=====

Enter Student Name: Kuhu
Enter Student ID: 1234
Enter Mood (1-5): 1
Enter Stress Level (1-5): 4
Enter Sleep Hours: 4
Enter Study Hours: 4

===== RESULT =====
Student Name   : Kuhu
Student ID    : 1234
Mood          : 1
Stress Level  : 4
Sleep Hours   : 4
Study Hours   : 4
Wellness Score : 40
Status       : Fair

Suggestions:
- Try doing something you enjoy.
- Take regular breaks while studying.
- Try to sleep at least 7-8 hours.

Would you like to check another student? (Y/N): N

Thank you for using the Student Mental Wellness Check-in System!
```

6. Discussion

The results show that the program responds differently depending on the student's input. Students with healthier values receive a higher wellness score, while students with several concerning values receive a lower score.

The project also demonstrates how functions can make a program more organized. The `calculateScore()` function is responsible for calculating the score, while the `suggestions()` function is responsible for providing suggestions.

The input validation also prevents the user from entering an invalid mood or stress value outside the 1–5 range.

7. Challenges and Limitations Challenges

We faced a few challenges while making the project:

- **Mood and stress range:** We had to make sure the user could only enter a value from 1 to 5. We used a while loop to check this.

- Taking the student's name: We had to learn how to take a name as input using `fgets()`.
- Using `strcpy()`: We learned how to store the names of the preset students.
- Calculating the score: We had to decide how many points to reduce for low mood, high stress, less sleep, and long study hours.
- Using functions: We had to understand how to send the student's information to the `calculateScore()` and `suggestions()` functions.

Limitations

- The program only checks mood, stress, sleep, and study hours.
- The scoring system is simple.
- It does not save student information after the program ends.
- The suggestions are basic.

8. Conclusion

The Student Mental Wellness Check-in System successfully demonstrates how basic C programming concepts can be used to create a simple system related to student life.

Through this project, we practiced functions, variables, conditional statements, loops, strings, and input handling. The program can calculate a wellness score, display a wellness status, and provide suggestions based on the student's information.

Overall, the project helped us understand how the programming concepts learned in class can be combined to create a complete working program.

9. Future Work

In the future, we could improve the system by:

- Adding more wellness questions.
- Saving student records using file handling.
- Adding more detailed suggestions.
- Creating a graphical user interface.
- Adding a feature to compare wellness results over time.
- Making the scoring system more detailed.

10. References

For this project, we mainly used our **class notes** and the programming concepts taught during the course. We also used the online C compiler and CS50.dev for writing, compiling, and testing the program.

References:

1. Course Class Notes, Computational Thinking and Programming.

2. CS50.dev — Development environment used for writing and testing the C program.
3. Online C Compiler — Used for testing C code.
4. https://www.cprogramming.com/tutorial/c/lesson9.html?utm_source=chatgpt.com